

小型微型计算机系统
Journal of Chinese Computer Systems
ISSN 1000-1220,CN 21-1106/TP

《小型微型计算机系统》网络首发论文

题目：多提示策略驱动的安卓恶意软件多标签检测
作者：白裕德，梁鑫淼，董敏弘，许光全，胡帅，余韦，王萌，何文颖
DOI：10.20009/j.cnki.21-1106/TP.2026-0180
收稿日期：2026-04-23
网络首发日期：2026-08-25
引用格式：白裕德，梁鑫淼，董敏弘，许光全，胡帅，余韦，王萌，何文颖．多提示策略驱动的安卓恶意软件多标签检测[J/OL]．小型微型计算机系统.
https://doi.org/10.20009/j.cnki.21-1106/TP.2026-0180



网络首发：在编辑部工作流程中，稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定，且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式（包括网络呈现版式）排版后的稿件，可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定；学术研究成果具有创新性、科学性和先进性，符合编辑部对刊文的录用要求，不存在学术不端行为及其他侵权行为；稿件内容应基本符合国家有关书刊编辑、出版的技术标准，正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性，录用定稿一经发布，不得修改论文题目、作者、机构名称和学术内容，只可基于编辑规范进行少量文字的修改。

出版确认：纸质期刊编辑部通过与《中国学术期刊（光盘版）》电子杂志社有限公司签约，在《中国学术期刊（网络版）》出版传播平台上创办与纸质期刊内容一致的网络版，以单篇或整期出版形式，在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊（网络版）》是国家新闻出版广电总局批准的网络连续型出版物（ISSN 2096-4188，CN 11-6037/Z），所以签约期刊的网络版上网络首发论文视为正式出版。

网络首发时间：2026-08-25 14:23:04　网络首发地址：https://link.cnki.net/urlid/21.1106.TP.20260825.1316.004

DOI：10.20009/j.cnki.21-1106/TP.2026-0180

多提示策略驱动的安卓恶意软件多标签检测

白裕德[1]，梁鑫淼[2]，董敏弘[3]，许光全[3]，胡帅[4]，余韦[5]，王萌[2]，何文颖[6]

[1]（天津工业大学 软件学院，天津，300387）

[2]（天津工业大学 计算机科学与技术学院，天津，300387）

[3]（天津大学 网络安全学院，天津，300072）

[4]（国网新疆电科院，乌鲁木齐，830000）

[5]（浙江越秀外国语学院，浙江 绍兴，312000）

[6]（河北工业大学 人工智能与数据科学学院，天津，300401）

E-mail：hwying1234@hebut.edu.cn

摘　要：针对安卓恶意软件多标签检测中的噪声敏感与少样本挑战，本文提出多提示策略协同的安卓恶意软件多标签检测框架，系统比较链式思维（Chain-of-Thought，CoT）、回溯式推理（Step-back）与树状思维（Tree-of-Thought，ToT）三种提示策略的性能差异。框架包含 3 个核心模块：基于谱聚类的降噪样本选择模块、融合标签语义与少样本示例的多策略提示工程模块，以及适配 0.5B 至 7B 参数规模的模型推理模块。在 Drebin、VirusShare 和 anonymousCERT 这 3 个数据集上的实验结果表明：CoT 策略在 7B 模型上的 F1-Score 比 Step-back 和 ToT 平均高出 8%~12%，综合性能最优；3B 模型在性能与成本之间取得最佳平衡，MS-ACC 较 0.5B 提升 15%且接近 7B 水平。上述发现揭示了提示策略与模型规模之间的性能关联规律，为资源受限环境下的检测系统配置提供了实践依据。

关键词：安卓恶意软件；多标签检测；大语言模型；链式思维；提示策略

中图分类号：TP393　　**文献标识码：**A

Multi-prompt Strategy Driven Android Malware Multi-Label Detection

Bai Yude[1]，Liang Xinmiao[2]，Dong Minhong[3]，Xu Guangquan[3]，Hu Shuai[4]，Yu Wei[5]，Wang Meng[2]，He Wenying[6]

[1]（School of Software, Tianjin University of Technology, Tianjin 300387,China）

[2]（School of Computer Science and Technology, Tianjin University of Technology, Tianjin 300387,China）

[3]（School of Cybersecurity, Tianjin University, Tianjin 300072,China）

[4]（State Grid Xinjiang Electric Power Research Institute, Urumqi 830000,China）

[5]（Zhejiang Yuexiu University of Foreign Languages, Shaoxing 312000,China）

[6]（School of Artificial Intelligence and Data Science, Hebei University of Technology, Tianjin 300401,China）

Abstract: Multi-label Android malware detection faces challenges from noisy labels and limited samples. This paper proposes a multi-prompt collaborative framework for Android malware multi-label detection, systematically comparing Chain-of-Thought (CoT), Step-back, and Tree-of-Thought (ToT) prompting strategies. The framework comprises three core modules: a spectral clustering-based denoising sample selection module, a multi-strategy prompt engineering module integrating label semantics with few-shot examples, and a model inference module accommodating LLMs from 0.5B to 7B parameters. Experiments on Drebin, VirusShare, and anonymousCERT datasets demonstrate that CoT achieves 8% to 12% higher F1-Score than Step-back and ToT on the 7B model, while the 3B model offers the optimal performance-cost balance with a 15% MS-ACC improvement over 0.5B, approaching 7B-level performance. These findings reveal the performance interplay between prompt strategies and model scales, providing practical guidance for detection system deployment in resource-constrained settings.

Keywords: Android malware; multi-label detection; large language models; chain-of-thought; prompt strategy

收稿日期：2026-04-23 收修改稿日期：2026-06-26　基金项目：国家自然科学基金青年基金项目(62302148)资助。 作者简介：**白裕德**，男，1991 年生，博士，讲师，研究方向为软件与系统安全；**梁鑫淼**，男，2002 年生，硕士研究生，研究方向为软件与系统安全；**董敏弘**，男，1999 年生，博士研究生，研究方向为软件与系统安全；**许光全**，男，1979 年生，博士，教授，研究方向为软件与系统安全；**胡帅**，男，1987 年生，博士研究生，高级工程师，研究方向为发电、调相机组并网与运行控制工作；**余韦**，男，1989 年生，博士，副教授，研究方向为复杂网络分析；王萌，男，1989 年生，博士，讲师，研究方向为计算机视觉；**何文颖(通信作者)**，女，1990 年生，博士，副教授，研究方向为机器学习和生物信息学。

DOI：10.20009/j.cnki.21-1106/TP.2026-0180

0 引　言

随着移动互联网的快速发展，智能手机已成为人们获取信息的主要终端设备。Android 凭借约87%的市场份额在移动操作系统中占据主导地位。然而，其开放生态也使其成为恶意软件传播的重点目标。攻击者通过恶意应用实施非法扣费、资源滥用及隐私窃取等行为[1]，对用户与平台安全构成持续威胁。在此背景下，提升安卓恶意软件检测的准确性与稳定性，成为网络空间安全领域的重要研究方向。

针对安卓恶意软件检测问题，现有方法主要包括传统机器学习与深度学习两类。传统方法依赖人工特征工程，在标签关联复杂或行为模式交织时容易遗漏关键模式；深度学习方法（如 DL-Droid[2]、MobiTive[3]）通过自动特征提取提升性能，但在样本数量有限或标签存在噪声时表现明显下降[4, 5]。总体而言，现有方法难以同时兼顾噪声容忍性、少样本适应能力以及多标签语义建模，在复杂安全环境中仍存在局限。

近年来，大语言模型（LLMs）在代码理解与安全分析领域展现出较强的语义建模与推理能力，为安卓恶意软件多标签检测提供了新的技术路径。多标签检测要求模型同时识别恶意软件的家族归属、行为特征及风险等级等多个标签，本质涉及复杂语义关系建模与不完整信息推理。然而，将 LLMs 直接应用于该任务并不能保证稳定效果，其性能高度依赖模型规模与提示策略设计。

在此背景下，当前研究面临 3 个关键挑战。第一，模型规模与任务适配性尚不明确。小参数模型（如 0.5B）资源消耗低但能力有限，大参数模型（如 7B）性能更强却成本更高，两者在多标签检测中的适用边界缺乏系统验证。第二，提示策略效果存在差异。CoT 强调线性推理，回溯式推理（Step-back prompt，Step-back）强调反思校正，Tree-of-Thought（ToT）强调多路径探索，不同策略在多标签场景中的稳定性与泛化能力尚未形成共识。第三，数据分布与标签质量影响机制复杂。不同数据集（如专家核验的 anonymousCERT、含噪声的 VirusShare）在标签可靠性与样本分布上差异显著，其与模型规模及提示策略之间的交互规律仍有待系统分析。

针对上述问题，本文在前期 LeoDroid[6]框架基础上设计了整合多提示策略的扩展框架。框架包括 3 个核心模块：通过谱聚类构建高质量样本选择模块以降低噪声干扰；构建融合标签语义、少样本示例与 CoT、Step-back、ToT 这 3 种策略的提示工程模块；构建适配 0.5B、3B、7B 参数规模 LLMs 的模型推理模块，实现跨规模性能对比。该框架在统一结构下系统评估模型规模、提示策略与数据分布的影响，从而提升多标签推理稳定性。

为验证框架有效性，本文在 anonymousCERT、Drebin 与 VirusShare3 个数据集上开展实验。结果表明，CoT 在 7B 模型上的 F1-Score 比 Step-back 和 ToT 平均高 8%-12%；在模型规模对比中，3B 模型在性能与成本之间取得更优平衡，其 MS-ACC 比 0.5B 高 15%，且接近 7B 模型水平。上述结果揭示了提示策略与模型规模之间的适配规律。。

本文的主要贡献如下：1）系统比较 CoT、Step-back 与 ToT 在安卓恶意软件多标签检测中的性能差异；2）提出基于模型规模的提示策略选择准则；3）通过多数据集实验验证框架在噪声与少样本环境中的稳定性，为 LLM 在安全检测领域的应用提供实践依据。

1 相关工作

1.1 安卓恶意软件检测与多标签分类

恶意软件[7]是攻击者实施网络攻击的重要手段，例如利用恶意软件进行远控或者发起勒索，为有效保护计算机网络和系统免遭破坏，恶意软件检测一直以来都是网络空间安全领域热点研究方向之一。在安卓恶意软件检测领域，研究者已提出多种方法，从静态与动态分析到深度学习模型，不断提升检测的准确性与泛化能力。Alzaylaee 等人提出的 DL-Droid 系统结合了动态行为分析与有状态输入生成策略，能够在真实设备上运行，有效提升检测能力[2]。在此基础上，Feng 等人开发了 MobiTive，通过性能驱动方式自动提取关键二进制特征，实现了跨设备的移动端实时检测[3]。为进一步增强模型的可解释性，Iadarola 等人引入了 Grad-CAM 热图机制，利用视觉方式揭示深度模型的判别依据[8]。此外，APIGraph 框架通过构建 API 语义图捕捉函数间潜在联系，从而增强了对恶意行为演化的刻画能力[9]；Amer 等人则在其检测框架中融合统计特征、上下文建模和图挖掘等多维度手段，实现了对良性与恶意行为的高效区分[10]。A1Sobeh 等人提出的 TAML（Time-Aware Machine Learning）系统进一步将时间因素纳入检测流程，以增强模型

对行为序列变化的感知力[11]。

然而，随着恶意软件行为呈现出日益复杂和混合的趋势，研究者逐渐认识到单标签分类模型已难以满足实际检测需求。尤其在安卓平台上，同一恶意软件样本往往同时具备窃取信息、远程控制、广告注入等多种行为特征，多标签分类模型因此成为提升检测精度与实用性的关键方向[12]。Bai 等人率先提出基于连体网络的多标签家族分类模型，将该任务建模为对比学习问题，有效缓解了样本稀缺场景下的性能退化[13]。IAFDroid 则通过结合静态分析与污点传播技术，对安卓应用间的协同攻击行为进行识别，从而拓展了模型对跨应用威胁的识别能力[14]。在行为特征提取方面，Amer 等人整合 API 调用序列、权限模式与语义上下文特征，构建深度学习模型以实现细粒度的多标签判别[15]。Qiao 等人进一步引入主动学习机制，基于报告分区策略在大规模未标注样本中自动识别潜在标签关系，在小样本场景下提升了标签覆盖率与泛化能力。

尽管上述方法在检测与分类能力方面取得了一定进展，但在实际部署中仍存在显著挑战。一方面，DL-Droid 的动态分析需要大量系统资源，MobiTive 在复杂特征提取阶段容易形成性能瓶颈，TAML 和 APIGraph 则对时间序列建模与图结构分析提出较高计算要求。另一方面，由于恶意软件家族的不平衡分布与持续演化，现有方法在应对新兴类别时普遍存在适应能力不足的问题；同时，当前主流的行为分析技术多数依赖局部特征或单一行为通道，难以捕捉不同恶意行为模式之间的语义重叠与协同关系。此外，现有可解释机制如 Grad-CAM 主要提供局部激活区域，缺乏对恶意行为链条的因果解析，在多标签恶意行为识别中显得尤为受限。

1.2 恶意软件下的噪音处理

噪音处理是恶意软件检测任务中一个普遍存在但又经常被忽视的因素。不过，重新搜索者已经越来越认识到它的影响[16, 17]。Li 等人[18]提出了 MalCertain，这是一个旨在提高基于深度神经网络(DNN)的安卓恶意软件检测模型性能的框架。考虑到嘈杂环境中概念漂移和对抗性示例带来的挑战，MalCertain 可估算预测性不确定性，以提高检测性能。通过分析特征分布，该框架确定了能有效区分正确和错误预测的不确定性估计方法。在此基础上，训练校正模型来校准 DNN 预测。此外，Yang 等人[19]引入了 URFEAL，通过抗噪采样策略减轻主动学习候选集中的冗余，从而提高数据选择的准确性和可靠性。这项工作提出了在噪声环境中增强主动学习算法稳定性的新方法。

1.3 基于 LLM 的提示词方法

近年来，大规模语言模型（LLMs）在自然语言理解与代码生成等任务中展现出强大的推理与上下文建模能力，研究者也开始探索其在安全检测领域的应用潜力[20, 21]。传统特征工程方法在跨平台、跨语言和小样本场景中表现受限，而基于 LLM 的方法为应对这些挑战提供了新思路。提示工程（Prompt Engineering，PE）是 LLM 对话模式的技术基础[22]。Liu 等人系统梳理了提示工程（Prompt Engineering）的发展脉络，提出多种即时学习（In-Context Learning, ICL）优化策略，并验证了输入模板与上下文示例设计对模型性能的显著提升[23]。在此基础上，Yu 等人提出 MalTracker 系统，将调用图分析与跨语言恶意行为表示结合，引入 LLM 模型进行上下文理解与特征迁移，在 Python、JavaScript 等多种语言环境中实现了统一的行为表示空间，有效提升了跨语言检测性能[24]。

在提示工程的发展中，回溯式推理逐渐受到关注。Shinn 等人提出的 ReAct 框架将模型输出拆分为“思考（Thought）”与“行动（Action）”阶段，通过显式反馈增强模型的推理调整能力[25]；Madaan 等人的 Self-Refine 模型则引入 Step-back 思维，允许模型在初次输出后进行“自我校正”，显著提升了开放式任务的准确率与稳定性[26]。这种“思考—反省—调整”的机制被证明在多标签任务中有助于减少类别遗漏与冗余标签生成，提高模型的 F1 表现和稳定性。

与此同时，树状思维框架为复杂任务提供了更具结构化的解法。Hulbert 等人提出的 ToT 方法通过构造多路径思维树并结合路径评分机制，引导模型在多个候选推理路径中进行分支与评估，从而实现发散—收敛的推理过程[27]。后续研究表明，ToT 在数学证明、逻辑游戏和代码生成等任务中较传统 CoT 表现出更高准确率与稳定性。例如，LongAlign 模型通过长上下文支持进一步优化了 ToT 在长序列任务中的表现。尽管 ToT 仍存在计算开销大和提示结构复杂等问题，但其启发式搜索框架在多标签分类与复杂安全检测任务中展现出极大潜力。

2 方法

2.1 问题表述

设训练数据集表示为$D=(x_i,y_i),i=1..n$，其中，$x_i \in R^d$表示第 i 个 Android 应用的特征向量，$y_i \in (0,1)^m$为其对应的多标签向量。此处，d表示特征维度，m为可能的恶意行为标签数量。

在存在噪声的场景下，设噪声比例为$\lambda \in [0,1]$，则假设数据集中约有 λ_n 个样本可能存在标签干扰或特征偏移。

通过引入核心集策略，本文将数据集划分为 k 个聚类簇，记作$C=C_j,j=1..k$。每个簇代表一类结构相似的样本集合，核心样本用于后续提示生成与知识引导。

本研究的目标是设计一个提示函数$f_\theta()$，其能够从特征向量x_i映射得到预测标签向量y_i。该函数通过利用核心簇中的代表样本，在提升分类准确性的同时，增强模型对噪声数据的稳定性。

2.2 整体框架

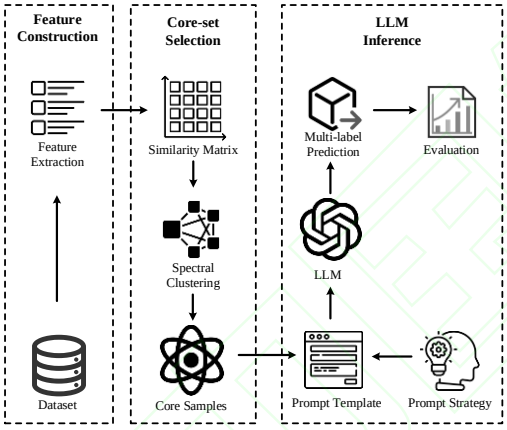


图 1 多提示策略协同的安卓恶意软件多标签检测框架

Fig.1 Framework of multi-prompt strategy driven android malware multi-label detection

如图 1 所示，工作流程包括以下关键步骤：

1) 特征构造。首先从原始的安卓应用数据集出发，进行基础且关键的特征提取。本阶段针对安卓应用的静态行为，主要从 API 调用、权限使用模式以及意图（Intent）结构 3 个核心维度进行特征解析。通过上述提取，将非结构化的原始应用转换为包含 531 个维度的结构化行为特征集。

2) 核心集构造。无论在何种数据分布下，高质量样本的选择都是保障大模型上下文学习效果的基础。该模块旨在通过降噪来构建可靠的少样本（Few-shot）示例集。首先，通过计算样本特征向量间的 JS 散度矩阵，并将其转换为相似度矩阵。随后，引入谱聚类（Spectral Clustering）算法对特征空间进行划分，将结构相似的恶意软件样本聚为同一簇。最后，通过计算簇内样本间的平均距离，提取各簇中心附近最具代表性的高质量样本作为核心集，从而有效降低了将噪声样本纳入训练与推理过程的风险。

3) 大模型推理。首先，在提示词设计上，系统将前置模块提取的核心样本转化为高质量的上下文示例，并融合专家身份设定、多标签语义以及特定的提示策略（如 CoT、Step-back 或 ToT），生成结构化的提示词模板。该文本旨在引导大语言模型在输入特征与分类决策之间建立严密的逻辑链条。随后，利用大模型进行推理。本框架灵活支持不同参数规模（如 0.5B、3B、7B 的 Qwen2.5、LongAlign 等）的预训练语言模型；模型在接收结构化提示后，自主执行多标签分类任务并输出目标应用的恶意行为标签集合。最后，为验证本方法的有效性，框架设计了层层递进的性能评估体系：先通过对比不同提示策略的绝对性能来确立最优方案；再通过探究模型规模与策略效果的交互关系来权衡性能与算力开销；最终在涵盖不同数据分布与噪声水平的真实数据集（Drebin、VirusShare、anonymousCERT）上展开综合测试，深入检验所提策略的鲁棒性与泛化能力。

2.3 高质量样本选择

为进一步提升样本选择策略在多标签任务下的代表性与稳定性，本文引入谱聚类（Spectral Clustering）方法，以替代传统 KNN 聚类算法[28-30]。谱聚类通过对样本之间的相似度矩阵进行特征分解，能够在非凸分布或复杂结构的数据空间中获得更合理的划分结果。

2.3.1 相似度矩阵构造

本研究直接加载预计算的 JS 散度矩阵（Jensen-Shannon divergence matrix）[31]作为样本间差异度的量化指标，其元素表示样本间的差异程度（值越大，差异越大）。为适配谱聚类对相似度的需求，将 JS 散度转换为相似度矩阵$S \in R^{n \times n}$，转换方式为：

$$S_{ij}=1-\mathrm{JS}(x_i,x_j) \tag{1}$$

其中$\mathrm{JS}(x_i,x_j)$为样本x_i与x_j的 JS 散度值，$S_{ij} \in (0,1)$（值越大，样本越相似）。

对相似度矩阵S进行归一化处理（映射至[0,1]区间），得到：

$$S_{norm,ij} = \frac{S_{ij} - \min(S)}{\max(S) - \min(S)} \tag{2}$$

2.3.2 谱聚类核心流程

在谱聚类中，基于归一化后的相似度矩阵S_{norm}构建图结构，具体步骤如下：

1) 图拉普拉斯矩阵构建：采用 K 近邻（K-Nearest Neighbors）策略从S_{norm}中提取局部相似性，构建图拉普拉斯矩阵 L，以强化样本间的局部关联刻画。
2) 特征分解与嵌入空间：对 L 进行特征值分解，提取前 K 个最小特征值对应的特征向量，构建嵌入矩阵$U \in R^{n\times K}$，实现样本从原始空间到低维嵌入空间的映射。
3) K-Means 聚类：在嵌入空间中，采用 K-Means 算法（目标函数为最小化簇内平方和）对样本进行聚类，得到聚类标签：

$$\min_{C_1,C_2,\ldots,C_K} \sum_{k=1}^{K} \sum_{x_i \in C_k} \left\| \widehat{U}_i - \mu_k \right\|^2 \tag{3}$$

其中$\widehat{U}_i$为嵌入空间中的样本向量，μ_k为第 k 个簇的中心，C_k为第 k 个聚类簇。

2.3.3 自适应聚类簇数选择

为确保聚类结果的稳健性，本文采用一种基于谱聚类的自适应聚类簇数选择方法。首先以归一化相似度矩阵 Snorm 为输入，设定候选聚类数范围为 K∈[2,9]。针对每一个候选聚类数 K，基于 Snorm 构建 k-近邻相似度图并执行谱聚类，通过 K-Means 对谱嵌入后的低维特征进行聚类，得到对应的聚类标签。

随后，为定量评估不同聚类数下的聚类质量，分别计算平均轮廓系数（Silhouette）与 Calinski–Harabasz 指标。其中，轮廓系数用于衡量样本在簇内的紧凑性与簇间的分离度，其定义为

$$\text{silhouette}(i) = \frac{b(i) - a(i)}{\max\big(a(i), b(i)\big)} \tag{4}$$

其中$a(i)$表示样本 i 与同簇样本之间的平均距离，$b(i)$表示样本 i 与最近异簇样本之间的平均距离。对所有样本的轮廓系数取平均，得到该聚类结果对应的平均轮廓系数。Calinski–Harabasz 指标用于评价聚类结构的整体判别能力，其定义为

$$calinski = \frac{\text{SSB}/(K-1)}{\text{SSW}/(n-K)} \tag{5}$$

其中 SSB 和 SSW 分别表示簇间平方和与簇内平方和，n 为样本总数。

为消除不同评价指标量纲差异的影响，对平均轮廓系数与 Calinski–Harabasz 指标分别进行归一化处理，并将其线性融合构建综合评分函数：

$$score = \text{silhouette}_{norm} + \text{calinski}_{norm} \tag{6}$$

最终选取综合评分最大的聚类数作为最优聚类簇数：

$$K_{opt} = \text{argmax}(score) \tag{7}$$

并基于该簇数重新执行谱聚类，得到最终的聚类标签L_{opt}。

2.3.4 核心样本提取

聚类完成后，从每个簇中提取核心样本：

对簇C_k，计算所有样本间的两两距离（intra_dist），得到每个样本与簇内其他样本的平均距离；

选取平均距离最小的前 50%样本作为该簇的核心点[32]（若簇内样本数>3），即：

$$x_{core} = \underset{x_j \in C_k}{\text{argmin}} \frac{1}{|C_k| - 1} \sum_{x_m \in C_k, x_m \neq x_j} d(x_j, x_m) \tag{8}$$

其中$\mathrm{d}(\mathrm{x_j}, \mathrm{x_m})$为样本$x_j$与$x_m$的距离；

累计核心样本，直至总数达到数据集规模的 50%，其余样本作为边界点。

2.4 提示设计

在 Leo-Droid 模型中[6]，有效的提示设计对优化分类性能和计算效率起着至关重要的作用。本模型处理 Android 应用特征，如 API 调用、权限和意图。得益于 LLM 对这些技术元素的固有理解，能够实现压缩输入格式，在不牺牲特征表示质量的情况下减少令牌的使用。

尽管 LLMs 拥有广博的知识库，但他们需要明确的指导才能正确地将特征与分类标签联系起来。本方法以成熟的研究方法为基础，将详细的标签定义和特征关系整合到提示结构中。这种结构化框架使模型能够在输入特征和目标标签之间建立稳健的联系，从而实现准确的多标签分类。本研究设计的模块化提示词结构模板如图 2 所示。

DOI：10.20009/j.cnki.21-1106/TP.2026-0180

You are a multi-label malware classification expert.
The following **categories** describe **different malicious behaviors**:
message : behaviors related to sending or receiving messages (e.g., SMS, email).
blackmail : behaviors related to fraud, threats, or unauthorized device control.
keep : behaviors related to maintaining background services or persistent access.
phone : behaviors related to phone calls or call monitoring.
ad: behaviors related to advertisements or ad-click activities.
internet : behaviors related to network communication, data transmission, or external services.

The following are several representative malware samples along with their corresponding labels.
Sample Name:
f3bcda140671049a060bef4fd0fc89cb99d4984a379a5fcccc74dd1331a0c4931
API Features:
getDefault, sendTextMessage, parse, init, read, append, get, readLine
Permission Features:
android.permission.CHANGE_CONFIGURATION,android.permission.INTERNET, android.permission.READ_PHONE_STATE,android.permission.RECEIVE_BOOT_COMPLETED,android.permission.RECEIVE_SMS,android.permission.SEND_SMS
Intent Features:
android.intent.category.LAUNCHER
Labels:
[blackmail, keep]
……

[Chain-of-Thought Prompt]
Please analyze the current malware sample step by step.
[Step-back Prompt]
First summarize the common behavioral characteristics reflected by the sample,and then infer its malicious behavior categories based on these characteristics.
[Tree-of-Thought Prompt]
Expert A (Semantic Expert): infers labels based on the functional semantics of APIs, permissions, and intents.
Expert B (Pattern Expert): performs matching-based reasoning using rules, templates, and co-occurrence patterns from labeled samples.
Expert C (Statistical Expert): conducts verification and auxiliary reasoning based on feature frequency, rare characteristics, and anomalous combinations.
The experts reason independently and evaluate each other's conclusions. Any expert with serious logical flaws is eliminated. The process iterates until a consensus is reached or a final classification result can be determined.

A malware sample is provided with the following features:
Sample Name:
c42b95620caaae1c03dfe1f2c756a3583cd0731d8522ee3275e9196db8027753
API Features:
getDeviceId, getSubscriberId, newWakeLock, init, getSystemService, read, append, close, get, readLine, BroadcastReceiver, WAKE_LOCK, onCreate, RECEIVE_BOOT_COMPLETED
Permission Features:
android.permission.ACCESS_NETWORK_STATE,android.permission.INTERNET,android.permission.MODIFY_PHONE_STATE,android.permissionREAD_PHONE_STATE,android.permission.RECEIVE_BOOT_COMPLETED,android.permission.WAKE_LOCK,android.permission.WRITE_APN_SETTINGS,android.permission.WRITE_EXTERNAL_STORAGE
Intent Features:
android.intent.action.BOOT_COMPLETED,android.intent.category.LAUNCHER
Question:
Which of the six malicious behavior categories does this sample belong to?

图 2 提示词模板示例图

Fig.2 Examples of prompt template

如图 2 所示，提示将从核心集策略中精心挑选的少量样本作为学习模板。每个模板都包含匿名样本标识符，以防止偏差，同时还包含完整的特征集及其相关的多标签类别。这种全面的背景为新样本的准确分类奠定了基础。

为了加强模型的分析能力，本文在提示设计中采用了 CoT 技术。CoT 模块超越了基本的标签和特征的定义，引导 LLM 完成结构化推理过程。通过分解特征标签这种方法可以减少因过于简化特征而产生的误差。

将 CoT 模块战略性地放置在提示语的末尾，可使 LLM 通过系统的中间推理建立在既定的上下文基础上。这种架构既提高了多标签分类的质量，又提高了模型的推理效率。

在原来研究的基础上，本文进一步引入 Step-back 推理机制以优化提示设计。不同于直接依赖特征与标签的映射关系，Step-back 方法首先将复杂的特征表示抽象到更高层次的语义层面。通过这一“后退一步”的概念化过程，模型能够更清晰地捕捉特征间的全局模式与潜在关系，从而减少因局部细节干扰而导致的分类误差。

具体而言，Step-back 模块会引导 LLM 将原始特征（如 API 调用、权限、意图等）先总结为若干中间语义单元，例如“用户隐私相关操作”或“网络通信行为”。这些抽象语义作为过渡层，再与目标标签进行关联。此种层级化的推理方式有效增强了模型对复杂特征空间的鲁棒性，避免了过拟合于低层次特征的风险。

与 CoT 不同，Step-back 更强调在推理初始阶段进行全局概括，使模型在分类过程中拥有更强的解释性和稳定性。这种方法在面对跨数据集迁移任务时尤其有效，因为抽象语义比原始特征更具泛化能力。

在 CoT 和 Step-back 的基础上，本文同时探索了 ToT 提示策略，以进一步提升 LLM 的推理深度与探索能力。ToT 不仅仅是线性推理链的扩展，而是将推理过程组织为树状结构，使模型能够在多条可能的推理路径中并行展开、选择与修正。

ToT 模块通过将特征与标签的关系拆解为多个候选推理分支，例如针对同一组 API 调用，模型

可能同时考虑“恶意下载行为”与“广告植入行为”的不同解释路径。随后，模型会通过内在评估机制选择最符合上下文与标签定义的分支，最终输出分类结果。

这种分支式推理使 LLM 能够避免单一路径推理的局限性，从而在处理多标签分类时具备更高的覆盖率和鲁棒性。尤其在标签之间存在交叉或重叠的场景下，ToT 能够通过并行分支探索保证更全面的分类决策。

在完成上述上下文构建与提示策略设定后，提示词的最终环节是引入待检测样本。大语言模型将基于前序构建的语义语境、少样本参考以及特定的提示策略（CoT、Step-back 或 ToT），对该未知样本的特征进行全面解析，并最终输出预测的恶意行为多标签分类结果。

3 实验

3.1 实验环境

因为考虑到 LLM 模型所需要的计算开销，本工作中的所有模型训练和测试都是在配备 Intel（R）Xeon（R）Platinum8336C 处理器和 NVIDIA GeForce RTX 4090 GPU 的 Ubuntu 服务器上进行的。本研究工作的主要编程语言为 Python。在本次实验中，LLM 模型使用的 torch 版本为 2.0.1，transformer 版本为 4.47.0，Cuda 版本为 11.8.89；同时选择 MEKAv1.9.2[33]作为机器学习多分类模型训练和评估模型的基本工具包，该工具包专为多标签分类问题提供一系列算法和评估指标。MEKA 是一个基于 WEKA 的开源项目。

3.2 数据集与实验设置

本研究使用 3 个主要数据集开展实验，涵盖多来源、多时间段的安卓恶意样本：

首先，anonymousCERT 数据集由安全专家核验，具备可靠的标签信息，作为本研究的核心标注数据来源。其次，本文使用了经典的 Drebin 数据集，该数据集收集了 2010 年 8 月至 2012 年 10 月间的 5560 个 Android 应用，涵盖 179 个恶意软件家族。第 3 个数据集来自 VirusShare，样本采集时间跨度为 2019~2022 年，反映了近年来恶意行为的演化趋势。

在对上述数据集进行深入分析的基础上，本文总结归纳出 6 类主要的恶意行为类型。特征提取方面，本文参考了已有研究中的主流方法，主要从以下 3 个维度构建特征：API 调用、权限使用模式、意图（Intent）结构。此外，本文进一步结合最新的技术文献与文档资料[34]，最终构建了一个包含 531 个维度的行为特征集，实现了对 Android 应用行为的全面刻画。

为保障评估的客观性与稳定性，从每个数据集中各选取 90 个高置信度、无噪声的样本作为测试集。为避免偏差干扰，few-shot 训练样本均经过匿名化处理，且保持各类标签分布的均衡性。在模型对比中，所有基于大语言模型（LLM）的方法均使用一致的 Prompt 构造模板，确保实验结果的公平性。

3.3 评价指标

3.3.1 Multi-Sample Accuracy

在多标签分类任务中，准确预测一个样本的所有标签具有天然的挑战性，尤其是在标注数据有限的情况下，要实现标签完全匹配往往并不现实。已有研究指出，传统的分类方法在多标签场景中常常难以同时保持各标签的预测准确性。

为更合理地衡量模型在此类任务中的表现，本文提出了一种具备容错机制的评估指标——Multi-Sample Accuracy（MS-ACC）。该指标在保持判别性能力的同时，引入了适度宽松的评估准则，更符合多标签学习的实际需求。

具体而言，某个样本x_i被认为预测正确，需同时满足以下两个条件：

- 至少一个标签命中：模型预测出的标签集合中，至少包含一个真实标签，即存在$Y_{i,j}=1 \ \wedge \ Z_{i,j}=1$，其中$Y_{i,j}$表示真实标签，$Z_{i,j}$表示预测结果；
- 未出现全标签覆盖：模型不能将所有可能标签均预测为正类，即样本x_i至少存在一个未被选中的标签，防止模型通过“全选标签”策略获得虚假的高分。

该评估策略有效避免了“全标签”预测的投机行为，同时也为多标签分类任务提供了更具现实意义的性能衡量方式，特别适用于 few-shot 和标签稀疏场景下的稳定评估需求。

$$\begin{cases} C_i=\{Z_{i,j}|(Y_{i,j}=1 \wedge Z_{i,j}=1) \wedge \sum_{j=1}^{|L|} Z_{i,j}<|L|, j\in[1,|L|]\}, \\ C=\{x_i|\exists C_i \ni 1, x_i\in D, i\in[1,|D|]\}, \\ \mathrm{MS-ACC}=\dfrac{|C|}{|D|} \end{cases} \tag{9}$$

3.3.2 Hamming Loss

Hamming Loss 是多标签分类任务中常用的标

签级评估指标，用于衡量模型在标签分配上的整体误差率。该指标通过统计所有样本中被错误预测的标签数量，并将其与标签总数进行归一化处理，从而反映模型在标签维度上的预测准确性。

具体而言，Hamming Loss 表示预测结果与真实标签之间的不一致程度。其数值越小，代表模型预测与真实标签集的匹配度越高，即标签分配更加精确。因此，Hamming Loss 的降低，通常意味着模型在多标签任务中的整体性能提升，尤其适用于衡量在标签粒度层面的细致预测能力。

3.3.3 Zero-One Loss

Zero-One Loss 是一种从样本级别评估模型性能的指标，用于衡量在多标签分类任务中，模型对整个标签集合预测是否完全正确的能力。其计算方式为：统计在所有样本中，至少有一个标签预测错误的样本所占比例。

与标签级指标（如 Hamming Loss）不同，Zero-One Loss 要求模型对每个样本的标签组合进行完全准确的预测，否则该样本即被视为预测失败。因此，该指标更加严格，对整体分类准确性提出更高要求。

数值上，Zero-One Loss 越低，表明模型在样本级的标签组合预测上越精确，即更大比例的样本其标签集合被完全预测正确，反映了模型在整体判断上的稳定性和完整性。

3.3.4 F1 Score

F1 Score 是衡量模型分类性能的核心指标之一，广泛应用于多标签和不均衡数据任务中。本研究采用 F1 Score[35-38]作为主要的平衡性评估指标，它通过对精确率（Precision）与召回率（Recall）计算其调和平均数，综合反映模型的整体预测能力。

F1 Score 的高低体现了模型在“准确预测正类标签”与“全面覆盖所有真实标签”这两个维度上的表现。当模型在同时保持高精确率和高召回率的情况下，F1 Score 将达到较高水平，意味着其在实际应用中具备良好的实用性与可靠性。

得益于这种双重衡量机制，F1 Score 能够为评估模型在复杂多标签场景中的实际有效性提供有力支持。

3.4 基线方法

3.4.1 LLM model

- Qwen2.5[39]基础模型的选择遵循以下准则：1）规模梯度覆盖：选取 0.5B、3B 及 7B 这 3 个参数规模等级，分别对应轻量级、中等规模及大规模模型，以系统评估参数规模对提示策略效果的影响；2）架构兼容性：模型需支持 Transformer 架构与中文语义理解，具备指令跟随能力以适应结构化提示词输入；3）开源可获取：所选模型需开源发布以保障实验可复现性；4）上下文容量：模型需具备至少 8K tokens 的上下文窗口以容纳少样本示例与完整特征描述。基于上述准则，本文选取 Qwen2.5 和 LongAlign 作为基础模型。Qwen2.5 是 Qwen 系列的最新版本，相较于 Qwen2，在多个方面实现了显著提升。它包含多个基础和指令调优模型，参数范围从 5 亿到 72 亿。Qwen2.5 提升了编码、数学能力，并在指令跟随、长文本生成（最多 8K tokens）、结构化数据理解（如表格）等方面表现更佳。它支持最大 128K tokens 上下文，并可生成最多 8K tokens 的文本。此外，Qwen2.5 支持 29 种语言，适应性强，适用于多样化的应用场景。该模型基于 Transformer 架构，结合了 RoPE、SwiGLU 等技术，参数量为 7.61B，广泛用于长文本生成和指令跟随任务。
- LongAlign[40]：LongAlign 是针对长上下文对齐任务的基准模型。它通过引入 LongAlign-10k 数据集，该数据集包含 10,000 条长度为 8k~64k 的长指令数据，提供了大规模的训练数据支持。此外，LongAlign 探索了两种训练策略：打包（结合损失加权）和排序批处理，这些策略有助于提高模型在长上下文上的表现。为了评估模型在现实应用中的表现，LongAlign 还使用 LongBench-Chat 作为评估工具，LongBench-Chat 主要评估模型在 10k~100k 长度查询上的指令跟随能力。

3.4.2 机器学习方法

本研究参考了先前关于安卓恶意软件多标签分类的工作，采用了 MEKAv1.9.2 作为机器学习工具包。MEKA 是一个基于 WEKA 的开源框架，支持通过更换基本分类器（BC）和多标签分类器（MLC）算法的组合来进行灵活的配置[33]。在本研究中，选用了 CDN 作为基本分类器（BC），并结合 LMT 方法与 REPtree 算法作为多标签分类器（MLC）进行安卓恶意软件的多标签分类。

3.5 实验结果与分析

3.5.1 Prompt 策略对模型性能的影响

为探究不同 Prompt 策略在 Android 恶意软件家族检测任务中的有效性，本文在统一的 7B 参数模型上，选取了 3 种典型的 Prompt 策略：CoT、Step-back 和 ToT，分别在 3 个广泛使用的恶意软件数据集 Drebin、VirusShare 和 anonymousCRET 上进行性能对比。

如图 3 所示，在 Drebin 数据集上，CoT 策略在多数指标上表现最优，其 F1-Score 达到 0.835，显著高于 Step-back（0.564）和 ToT（0.563）。并且在 MS-ACC，CoT 分别取得 0.978 的成绩，说明其逐步推理机制能有效捕捉家族特征。Step-back 在 Hamming Loss 和 Zero-One Loss 上略优于 ToT，表明其具有一定的错误纠正能力，但在整体分类一致性上不如 CoT。ToT 在所有指标上均表现最差，说明其多路径推理机制在此任务中引入了噪声，降低了分类准确性。

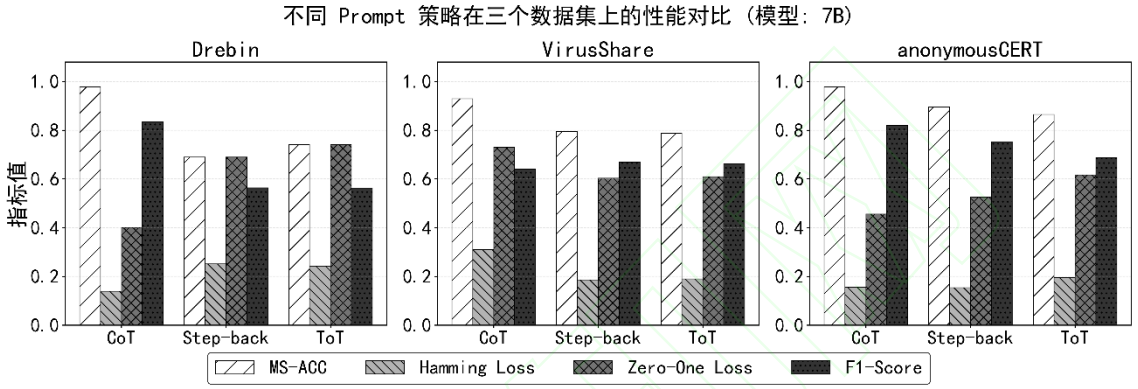


图 3 不同 Prompt 策略在 3 个数据集上的性能对比（模型:7B）

Fig.3 Performance comparison of different prompt strategies across three datasets (Model: 7B)

如图 3 所示，VirusShare 数据集上的结果与 Drebin 基本一致，CoT 依旧保持领先，F1-Score 为 0.64。Step-back 在 Hamming Loss 上表现最佳，说明其在处理高噪声数据时具有一定优势。ToT 在此数据集上表现略有提升，但仍显著低于 CoT 和 Step-back，尤其是在 Zero-One Loss 上明显偏高，表明其容易完全误判某些样本。

如图 3 所示，在 anonymousCERT 数据集上，CoT 与 Step-back 的表现较为接近，F1-Score 分别为 0.82 和 0.753，且均显著优于 ToT（0.687）。CoT 在 MS-ACC 上依旧领先，而 Step-back 在降低误报率（Hamming Loss）方面略有优势。ToT 在此数据集上的表现进一步验证了其不适用于家族分类任务的结论。

综合以上 3 个数据集的实验结果可以得出：CoT 策略在恶意软件家族检测中综合表现最优，其逐步推理机制能有效提升模型在 F1-Score 和 MS-ACC 等关键指标上的性能；Step-back 策略在降低误分类（Hamming Loss、Zero-One Loss）方面效果显著，适用于对误报敏感的应用场景；而 ToT 策略则因多路径推理引入噪声和不确定性，在所有数据集上均表现不佳。总体而言，CoT 策略在不同数据集上的一致性表现验证了其作为恶意软件家族检测任务的可靠选择。

3.5.2 模型规模对模型性能的影响

为探究模型参数规模对恶意软件家族检测性能的影响，本文在 3 个数据集（VirusShare、anonymousCRET 和 Drebin）上测试了 3 种不同参数规模的模型（0.5B、3B 和 7B），并结合 3 种 Prompt 策略（CoT、Step-back、ToT）进行性能评估。

图 4 和图 5 分别展示了 3 个数据集中不同模型规模下各 Prompt 策略的性能对比。总体而言，模型参数规模的增加显著提升了所有策略的性能表现，尤其是在从 0.5B~3B 的跃迁过程中，各指标提升最为明显。

如图 4、图 5 所示，在 VirusShare 数据集上，CoT 策略在 7B 模型上取得了最佳的整体性能（MS-ACC：0.93，F1-Score：0.64），显著优于 Step-back 和 ToT 策略。值得注意的是，Step-back 在 3B 和 7B 模型上均表现出最低的 Hamming Loss，说明其在减少错误分类方面具有持续优势。ToT 策略在所

有规模下性能均不及 CoT，且其 Zero-One Loss 较　　高，表明存在较多的完全误分类样本。

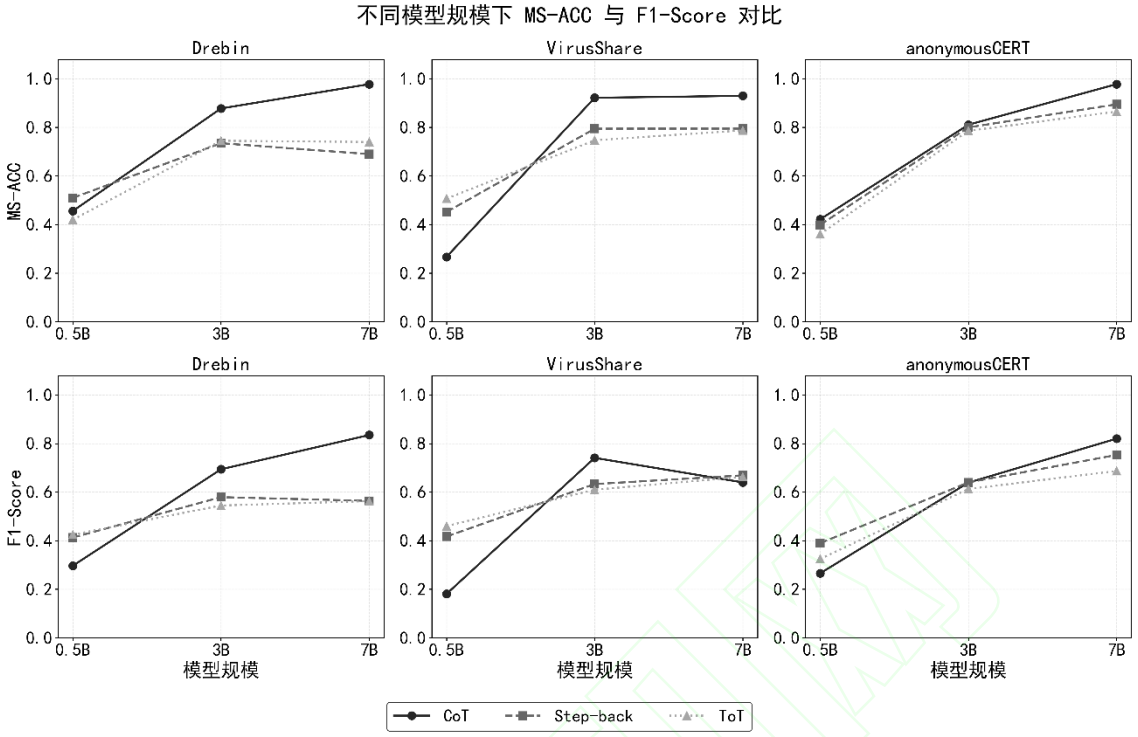


图 4 不同模型规模下 MS-ACC 与 F1-Score 对比

Fig.4 Comparison of MS-ACC and F1-Score across different model scales

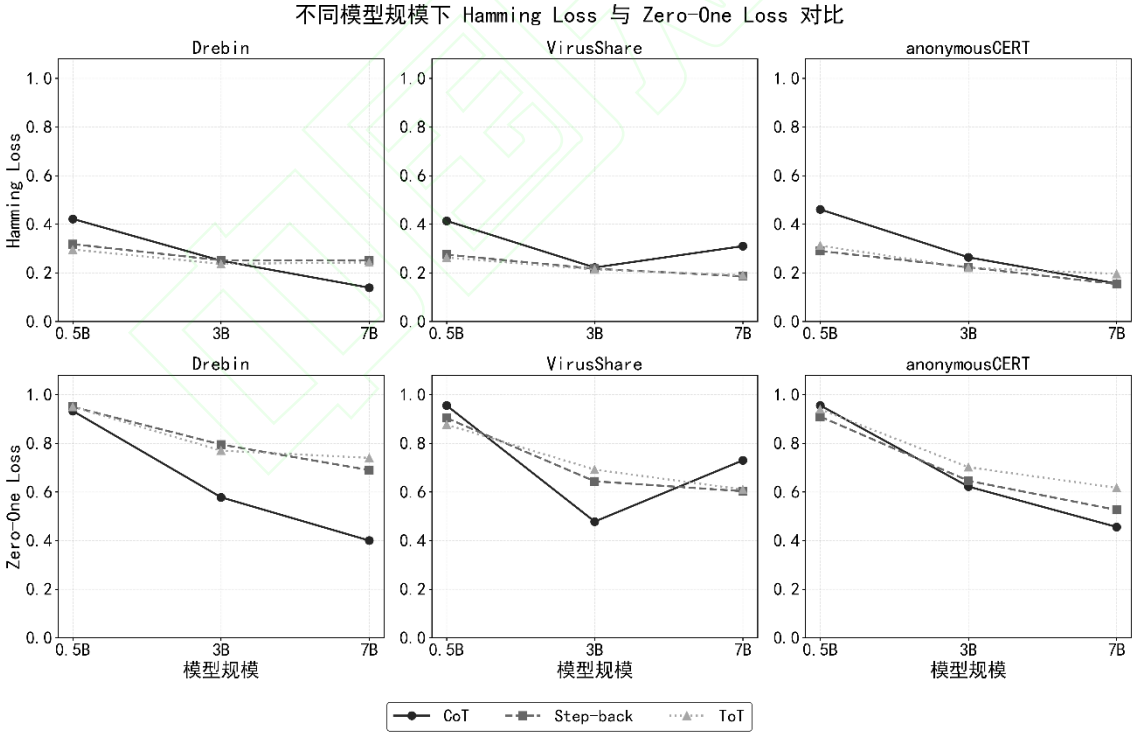


图 5 不同模型规模下 Hamming Loss 与 Zero-One Loss 对比

Fig.5 Comparison of Hamming Loss and Zero-One Loss across different model scales

如图 4、图 5 所示，在 anonymousCERT 数据　　集上，CoT 策略同样在 7B 模型上表现最优（MS-

ACC：0.978, F1-Score：0.82）。Step-back 策略在错误率控制上依然表现良好，但其分类准确性随模型规模增长幅度不及 CoT。ToT 策略性能依然垫底，且其提升幅度随模型规模增长最小。

如图 4、图 5 所示，在 Drebin 数据集上，所有策略在 7B 模型下均达到最佳性能。CoT 策略的 F1-Score 高达 0.835，显著领先。Step-back 在 3B 和 7B 模型上均保持较低的 Hamming Loss，进一步验证了其在错误控制方面的有效性。ToT 策略虽随模型规模提升有所改善，但仍无法与其他两种策略媲美。

综上所述，模型参数规模的扩大对提升恶意软件家族检测性能具有显著积极作用，尤其从 0.5B 到 3B 的提升幅度最大。CoT 策略在大多数情况下能充分利用模型增长带来的能力提升，尤其在分类准确性（MS-ACC、F1-Score）上表现最佳。Step-back 策略在控制错误率（Hamming Loss、Zero-One Loss）方面表现稳定且优异，适用于对误报敏感的场景。ToT 策略在所有数据集和模型规模下均表现不佳，不推荐用于此类任务。综上所述，选择足够大的模型（如 7B）并结合 CoT 策略，可在多数情况下获得最优的综合检测性能。

3.5.3 数据集差异对模型鲁棒性的影响

为系统评估不同数据分布对模型泛化能力与鲁棒性的影响，本实验选取了 3 个在数据分布、噪声水平及恶意软件家族界定清晰度方面具有显著差异的 Android 恶意软件数据集：Drebin、VirusShare 和 anonymousCERT。在统一的 7B 参数模型上，固定提示策略为表现最优的 CoT，测试其在不同数据集分布下的性能表现。评估指标涵盖分类准确性（MS-ACC、F1-Score）与错误率（Hamming Loss、Zero-One Loss），旨在从性能保持性与抗干扰性两个维度，综合评价 CoT 策略在不同数据环境下的适应能力。

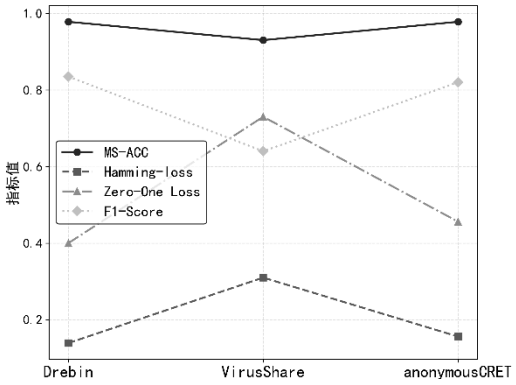


图 6 CoT 策略在不同数据集上的鲁棒性对比（模型:7B）

Fig.6 Robustness comparison of CoT strategy across different datasets (Model: 7B)

在数据分布规范、家族界限清晰的 Drebin 数据集上，模型表现出最优性能：MS-ACC 接近 0.98，F1-Score 达 0.835，Zero-One Loss 仅 0.4，显示出在良好数据条件下 CoT 的卓越性能与稳定性。

当模型应对结构复杂、噪声水平高、家族边界模糊的 VirusShare 数据集时，性能出现明显下降：MS-ACC 降至 0.93，F1-Score 大幅降低至 0.64，Zero-One Loss 升高至 0.73。这表明即使是最优策略 CoT，其表现依然会受到高噪声与分布复杂性的显著影响，凸显了数据质量在鲁棒性评估中的重要性。

在标签质量高但包含更多现代恶意软件变种的 anonymousCERT 数据集上，模型表现介于两者之间：MS-ACC 达到 0.978，F1-Score 为 0.82，但 Zero-One Loss 升至 0.456，说明在高质量标注下，复杂分布仍对模型判别能力提出了挑战。

实验结果表明，固定最优提示策略（CoT）后，数据集差异依然是评估模型鲁棒性的关键因素：

Drebin 作为经典基准，代表了模型的性能上限；

VirusShare 由于其高噪声与模糊家族界限，是检验模型抗干扰能力的有效场所；

anonymousCERT 通过高质量标签与复杂变种，适用于评估模型应对新型威胁的泛化能力。

总体来看，CoT 策略在优质数据（如 Drebin、anonymousCERT）上能稳定发挥优势，但其性能随数据复杂度增加而出现波动。这表明在恶意软件检测应用中，即便采用最优提示策略，模型的实用性不仅取决于峰值性能，更取决于其在多样化数据环

境下保持稳定表现的能力。

4 讨论

本研究通过 3 组实验系统性地评估了不同提示策略（CoT、Step-back、ToT）、模型规模（0.5B–7B）及数据集特性对多标签安卓恶意软件分类性能与鲁棒性的影响。实验结果表明，提示策略的效果高度依赖于模型能力与数据质量，三者共同构成一个相互影响的决策系统。

首先，在提示策略方面，CoT 凭借其结构化的逐步推理机制，在多数实验设定中表现出了最优的综合性能，尤其在 7B 模型上 F1-Score 领先幅度达 8%~12%。其优势在于推理路径清晰、确定性高，能够有效捕捉语义明确的家族特征。相比之下，Step-back 策略虽在错误率控制（如 Hamming Loss）方面表现稳定，尤其在高噪声数据中体现出较强的抗干扰能力，但其生成标签的保守倾向限制了召回率的提升；ToT 而 ToT 由于多路径推理引入噪声与不确定性，在所有实验条件下均表现不佳，说明其复杂的启发式搜索机制并不适用于此类需高确定性的安全检测任务。

模型规模的变化显著影响提示策略的有效性。从 0.5B 至 3B 的模型扩增带来性能的显著跃升，而从 3B 到 7B 的进一步提升则更多体现在 CoT 策略的稳定性优化上。值得注意的是，Step-back 和 ToT 在小模型中虽展现潜力，但随着参数增加，其性能增益逐渐饱和甚至出现回落，说明这些策略无法充分释放大模型的推理潜力，反而可能因冗余思维路径干扰主干判断。

再次，数据集差异对模型鲁棒性影响显著。Drebin 数据集标签清晰、噪声较低，3 种策略差距相对有限，代表了性能上限；VirusShare 数据集因噪声高、边界模糊，整体性能下降明显，其中 CoT 虽保持优势，但对数据质量的敏感性也显露出来；anonymousCERT 数据集则在标签准确的同时包含更多现代恶意软件变种，成为检验模型泛化能力的有效场景。整体而言，CoT 在优质数据中表现最佳，但在复杂数据条件下的性能波动需要关注。

综上所述，CoT 在当前任务中展现出最佳的实用性与综合性能，尤其适用于模型规模较大、数据质量较高的场景。若部署环境数据噪声显著或需极高分类稳定性，可优先考虑 Step-back 策略。ToT 在当前任务设置下适用性有限。未来的研究可着眼于提示策略的动态调度机制，以实现模型–策略–环境三者的自适应协同，进一步提升开放环境中的恶意软件检测效力。

5 总结

本研究系统性地探讨了基于多种提示策略的大语言模型在安卓恶意软件多标签检测任务中的应用效果与适用边界。面向当前恶意软件检测中存在的噪声敏感、少样本学习与多标签关联建模等挑战，本文构建了一种多提示策略协同的安卓恶意软件多标签检测框架，并在多个模型规模与数据集上开展了全面的实验验证。

主要研究内容与结论如下：

1）提出了一个多提示策略协同的安卓恶意软件多标签检测框架。该框架通过谱聚类算法从原始数据中提取高质量、低噪声的核心样本集，并在此基础上结合 CoT、Step-back 及 ToT 三种提示机制，有效引导大语言模型进行多标签推理，在提升检测性能的同时增强对噪声数据的鲁棒性。

2）通过在不同参数规模的模型（0.5B、3B、7B）与多样本特性的数据集（Drebin、VirusShare、anonymousCERT）上进行广泛实验，系统评估了各提示策略的性能与泛化能力。实验结果表明，CoT 策略在大多数实验设定中综合表现最优，尤其在 7B 模型上各类指标显著领先；Step-back 策略在高噪声环境中表现出良好的错误控制能力；而 ToT 策略因推理路径复杂且不确定性强，整体性能不佳。

3）研究还发现提示策略的效果高度依赖模型规模与数据质量。CoT 更适合大规模模型与清晰标注数据，Step-back 对噪声数据具备更强适应能力。模型参数从 0.5B 增至 3B 时性能提升最为显著，进一步扩展至 7B 则主要带来稳定性的增强。

本研究的实践意义在于为资源受限环境下的恶意软件检测系统提供了提示策略选择与模型配置的参考依据，强调在实际应用中需结合数据质量、计算资源与性能要求进行多维权衡。尽管当前框架在多个场景中表现良好，面对持续演变的恶意软件威胁，提示策略与模型结构的协同优化仍值得深入研究。

未来工作可从以下几个方向展开：一是探索提示策略的动态融合与调度机制，以适应不同数据分布和检测场景；二是引入外部知识或元学习机制，提升小样本下的快速适应与泛化能力；三是进一步优化噪声感知与样本清洗模块，构建更具鲁棒性的恶意软件表征学习框架。

References:

[1] LI H, WU B, ZHU G, et al. Overview of Android malware adversarial sample attack techniques [J]. Chinese Journal of Software, 2025, 36(6): 2683-2712.

[2] Alzaylaee M K, Yerima S Y, Sezer S. DL-droid: deep learning based android malware detection using real devices [J]. Computers & Security, 2020, 89: 101663.

[3] Feng R, Chen S, Xie X, et al. A performance-sensitive malware detection system using deep learning on mobile devices [J]. IEEE Transactions on Information Forensics and Security, 2020, 16: 1563-1578.

[4] Jia J, Feng X, Yu H. Few-shot classification via efficient meta-learning with hybrid optimization [J]. Engineering Applications of Artificial Intelligence, 2024, 127: 107296.

[5] Guo Q, Xie X, Liu S, et al. Intention is all you need: refining your code from your intention[C]//Proceedings of the IEEE/ACM 47th International Conference on Software Engineering, 2025: 728-728.

[6] Dong M, Liu L, Guo Q, et al. LeoDroid: an LLM-based few-shot multi-label detection for Android malware[C]//Proceedings of the IEEE Security and Privacy Workshops, 2025: 294-306.

[7] LIU H, TIAN Z H, QIU J, et al. A review of malware detection for small sample sizes [J]. Chinese Journal of Software, 2024, 35(8): 3785-3808.

[8] Iadarola G, Martinelli F, Mercaldo F, et al. Towards an interpretable deep learning model for mobile malware detection and family identification [J]. Computers & Security, 2021, 105: 102198.

[9] Zhang X, Zhang Y, Zhong M, et al. Enhancing state-of-the-art classifiers with api semantics to detect evolved android malware[C]// Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, 2020: 757-770.

[10] Amer E, Zelinka I, El Sappagh S. A multi-perspective malware detection approach through behavioral fusion of api call sequence [J]. Computers & Security, 2021, 110: 102449.

[11] Alsobeh A M,Gaber K,Hammad M M,et al. Android malware detection using time-aware machine learning approach [J]. Cluster Computing, 2024, 27(9): 12627-12648.

[12] Niu Z, Guo W, Xue J, et al. A novel anomaly detection approach based on ensemble semi-supervised active learning (ADESSA) [J]. Computers & Security, 2023, 129: 103190.

[13] Bai Y, Xing Z, Li X, et al. Unsuccessful story about few shot malware family classification and siamese network to the rescue[C]//Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering,2020: 1560-1571.

[14] Wang B, Yang C, Ma J. Iafdroid: demystifying collusion attacks in Android ecosystem via precise inter-app analysis [J]. IEEE Transactions on Information Forensics and Security, 2023, 18: 2883-2898.

[15] Amer E, El Sappagh S. Robust deep learning early alarm prediction model based on the behavioural smell for android malware [J]. Computers & Security, 2022, 116: 102670.

[16] Xu J, Li Y, Deng R H. Differential training: a generic framework to reduce label noises for android malware detection[C]// Proceedings of the Network and Distributed System Security Symposium, 2021.

[17] Liang J, Guo W, Luo T, et al. Fare: enabling fine-grained attack categorization under low-quality labeled data[C]//Proceedings of the Network and Distributed System Security Symposium, 2021.

[18] Li H, Xu G, Wang L, et al. Malcertain: enhancing deep neural network based

Android malware detection by tackling prediction uncertainty[C]//Proceedings of the IEEE/ACM International Conference on Software Engineering, 2024: 1-13.

[19] Yang J, Ma S, Zhang Z, et al. Say no to redundant information: Unsupervised redundant feature elimination for active learning [J]. IEEE Transactions on Multimedia, 2024, 26: 7721-7733.

[20] Guo Q, Cao J, Xie X, et al. Exploring the potential of chatgpt in automated code refinement: an empirical study[C]// Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, 2024: 1-13.

[21] Guo Q, Li X, Xie X, et al. Ft2ra: a fine-tuning-inspired approach to retrieval-augmented code completion[C]// Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2024: 313-324.

[22] BA Z Z, ZHANG H, XIE Z H, et al. A review of engineering technologies for automated language model hints [J]. Journal of Frontiers of Computer Science and Technology, 2025, 19(12): 3122-3131.

[23] Liu P, Yuan W, Fu J, et al. Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing [J]. ACM Computing Surveys, 2023, 55(9): 1-35.

[24] YU Z, Wen M, Guo X, et al. Maltracker: a fine-grained npm malware tracker copiloted by llm-enhanced dataset[C]//Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2024: 1759-1771.

[25] Yao S, Zhao J, Yu D, et al. React: synergizing reasoning and acting in language models[C]// Proceedings of the International Conference on Learning Representations, 2023.

[26] Madaan A, Tandon N, Gupta P, et al. Self-refine: iterative refinement with self-feedback [J]. Advances in Neural Information Processing Systems, 2023, 36: 46534-46594.

[27] Yao S, Yu D, Zhao J, et al. Tree of thoughts: deliberate problem solving with large language models [J]. Advances in Neural Information Processing Systems, 2023, 36: 11809-11822.

[28] Ng A, Jordan M, Weiss Y. On spectral clustering: analysis and an algorithm [J]. Advances in Neural Information Processing Systems, 2001, 14.

[29] Von Luxburg U. A tutorial on spectral clustering [J]. Statistics and Computing, 2007, 17(4): 395-416.

[30] Fan Y, Liu J, Tang J, et al. Learning correlation information for multi-label feature selection [J]. Pattern Recognition, 2024, 145: 109899.

[31] Endres D M, Schindelin J E. A new metric for probability distributions [J]. IEEE Transactions on Information Theory, 2003, 49(7): 1858-1860.

[32] Tu J, Mei G, Piccialli F. An improved Nyström spectral graph clustering using k-core decomposition as a sampling strategy for large networks [J]. Journal of King Saud University-Computer and Information Sciences, 2022, 34(6): 3673-3684.

[33] Read J, Reutemann P, Pfahringer B, et al. Meka: a multi-label/multi-target extension to weka [J]. Journal of Machine Learning Research, 2016, 17(21): 1-5.

[34] Chen S, Fan L, Meng G, et al. An empirical assessment of security risks of global android banking apps[C]// Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, 2020: 1310-1322.

[35] Xu G, Han Z, Gong L, et al. ASQ-FastBM3D: an adaptive denoising

framework for defending adversarial attacks in machine learning enabled systems [J]. IEEE Transactions on Reliability, 2022, 72(1): 317-328.

[36] Wang X, Dong Y, Jin D, et al. Augmenting affective dependency graph via iterative incongruity graph learning for sarcasm detection[C]//Proceedings of the AAAI Conference on Artificial Intelligence, 2023: 4702-4710.

[37] Zhang J, He R, Guo F, et al. Quantum interference model for semantic biases of glosses in word sense disambiguation[C]// Proceedings of the AAAI Conference on Artificial Intelligence, 2024: 19551-19559.

[38] Wang C, Yu H, Li X, et al. Dependency-aware microservice deployment for edge computing: a deep reinforcement learning approach with network representation [J]. IEEE Transactions on Mobile Computing, 2024, 23(12): 14737-14753.

[39] Yang A, Li A, Yang B, et al. Qwen3 technical report [J]. arXiv preprint arXiv:250509388, 2025.

[40] Bai Y, Lv X, Zhang J, et al. Longalign: a recipe for long context alignment of large language models[C]//Findings of the Association for Computational Linguistics: EMNLP 2024, 2024: 1376-1395.

附中文参考文献：

[1] 李珩, 吴棒, 龚柱, 等. 安卓恶意软件对抗样本攻击技术综述[J]. 软件学报, 2025, 36(6):2683-2712.

[8] 刘昊, 田志宏, 仇晶. 等. 面向小样本的恶意软件检测综述[J]. 软件学报, 2024, 35(8): 3785-3808.

[23] 巴泽智, 张辉, 谢铮涵. 等. 大语言模型自动化提示工程技术研究综述[J]. 计算机科学与探索 . 2025, 19(12): 3131-3122